One-to-One Record Matching Without Shared Keys: Integrating Manual Review into Updated Database Version

Biodiversity Data Integration

2025-12-10

Contents

1 Introduction

Durante el proceso de consolidación y depuración de datos, numerosas ocurrencias duplicadas (registros provenientes de una misma fuente e incorporados en distintos conjuntos de datos) no fueron detectadas por el algoritmo para de-duplicar datos, y debieron identificarse manualmente mediante planillas y QGIS. Asimismo, se realizaron mejoras puntuales en muchos registros, corrigiendo errores, completando información faltante, precisando coordenadas, añadiendo referencias asociadas, entre otros. Todas estas modificaciones se documentaron en la planilla `AtlasIctio_COMMENTS_2025_12.csv`.

En el caso de los duplicados, la columna `Duplicate` (no confundir con `duplicated`) registra dichas evaluaciones: los valores 1 y 0 indican ocurrencias a retener o excluir, respectivamente, mientras que las celdas vacías representan registros únicos. En la columna `Comment` se incluyeron comentarios diversos la mayoría de los cuales fueron pasados a otras columnas con el prefijo "." (por ejemplo, `.decimalLatitude`), donde un valor distinto de vacío indica el valor corregido del campo original correspondiente. `Comment` también contiene la palabra clave "remover" que indica que hay que remover la fila.

No obstante, el trabajo realizado sobre distintas versiones de la base consolidada provocó la pérdida del vínculo directo entre la tabla principal depurada y la tabla con anotaciones. Por ello, fue necesario restablecerlo mediante un protocolo de comparación exhaustivo entre tablas, implementando herramientas diagnósticas y verificaciones manuales en los casos dudosos.

Este documento describe el método empleado para reestablecer el vínculo uno-a-uno entre ambas tablas, asegurando correspondencias precisas y completamente trazables.

Principios clave del flujo de trabajo:

- Cada fila de `table1` debe coincidir de manera exacta con una única fila de `table2.`
- Las comparaciones se realizan exhaustivamente contra todas las filas de la otra tabla.
- Las coincidencias se establecen mediante combinaciones incrementales de campos clave previamente armonizados.
- El orden estratégico de estos campos favorece coincidencias correctas, conservadoras y no ambiguas.

2 Load Required Libraries

```
setwd("C:/Users/crist/Documents/GitHub/PEW-ProAp_Atlas Ictiogeografico_2025/Atlas_Ictio_project/table_join
rm(list=ls())

library(dplyr)
library(stringr)
library(ggplot2)
library(knitr)
library(tidyr)
```

3 Data importation

```
# Load data to start from here

X.filtered.dedup <- readRDS("./input/dataset_before_Remocion_manual_2025-12-03.rds")
```

```
CM <- read.csv("./input/AtlasIctio_COMMENTS_2025_12.csv", encoding = "UTF-8", sep = ",", colClasses = "cha
```

En esta tabla se excluyeron algunos bloques de datos desde la planilla: - (DatasetName == GBIF) & (institutionCode == iNaturalist). Ahora los datos de iNat tienen precedencia, y además se excluyenos los datos de iNat vía GBIF. - (datasetName == "gbif") & (str_sub(occurrenceID, 1L, 3L) == "mma"). Usamos la versión mejorada directamente del MMA, no la versión publicada en GBIF. - (datasetName == "subpesca-1") una lista de papers indexados en otras fuentes fue excluido. Ver planilla en la respectiva carpeta, y código de exclusión arriba.

First, fix truncated column names such that they match those of the database:

```
# Restore full column names from Maryse's table

full_names  <- names(X.filtered.dedup)
short_names <- names(CM)

new_names <- sapply(short_names, function(s) {
  # try exact prefix match: full name starts with the short name
  cand <- full_names[substr(full_names, 1, nchar(s)) == s]
  if (length(cand) == 1) {
    cand          # unique match → use full name
  } else {
    s             # no unique match → keep original
  }
}, USE.NAMES = FALSE)
# Check
# data.frame(old_name = colnames(CM),
#            full_name = new_names)
names(CM) <- new_names

CM <- CM %>%
  rename(
    institutionID   = institut_1,
    institutionCode = institutio,
    occurrenceID    = occurrence,
    occurrenceStatus = occurren_1,
```

```
    occurrenceRemarks = occurren_2)

# sort(colnames(CM))
# sort(colnames(X))
# CM %>% select(occurren_1, occurren_2) %>% distinct()
#
# X %>% select(occurrenceRemarks, occurrenceStatus) %>% distinct()
rm("full_names", "new_names", "short_names")
```

Merge (coalesce) two columns that were separated only due to a typo.

```
# Restore info from bibiographicCitation info

CM <- CM %>%
  mutate(
    across(c(bibliographicCitation, bibiograph),
           ~ str_trim(.) %>% na_if("") %>% na_if("NA")),
    bibliographicCitation = coalesce(bibliographicCitation, bibiograph)
  ) %>%  select(-bibiograph)
```

4 Data Preparation

4.1 Filter Source Data

Select columns from the comments table (CM) that can be used to match rows against the master databese, and all rows that were annotated requiring an action:

```
# Table1: rows with changes or exclusion requests
# Duplicate: "0" = exclude, "1" = keep, NA = no action needed
# Columns with "." prefix contain proposed changes
table1_raw <- CM %>%
  filter(
    Duplicate == "0" |
    !is.na(Comment) |
    if_any(starts_with("."), ~ !is.na(.x))
  ) %>%
  select(
    id_CC,
    Duplicate,
    decimalLatitude,
    decimalLongitude,
    canonicalName,
    year,
    eventDate,
    locality,
    samplingProtocol,
    individualCount,
    catalogNumber,
    datasetName,
    bibliographicCitation,
    institutionCode,
    occurrenceID,
    Comment,
    starts_with(".")
```

```
  )

cat("Table1 rows requiring action:", nrow(table1_raw), "\n")
```

```
## Table1 rows requiring action: 911
```

Note: There are several datasets that are not carried forward because no changes need to be made. These are:

```
setdiff( sort(unique(CM$datasetName)), sort(unique(table1_raw$datasetName)))
```

```
## [1] "Darwin Initiative Fish Atlas"
## [2] "iDigBio; Originaly provided by: AM"
## [3] "iDigBio; Originaly provided by: NMV"
## [4] "iDigBio; Originaly provided by: USNM, NMNH Extant Biology"
```

```
# Table2: consolidated master table after de-duplication
table2_raw <- X.filtered.dedup %>%
  select(any_of(c("id_CC", colnames(table1_raw))))

cat("Table2 total rows:", nrow(table2_raw), "\n")
```

```
## Table2 total rows: 6760
```

```
# Verify id_CC is unique
if (any(duplicated(table2_raw$id_CC))) {
  stop("ERROR: id_CC in table2 is not unique!")
}
```

4.2 Standardization Function

Reduce meaningless variation in key fields:

```
standardize_for_matching <- function(df) {
  df %>%
    mutate(
      # Coordinates: ensure numeric (no rounding to preserve precision)
      decimalLatitude  = as.numeric(decimalLatitude),
      decimalLongitude = as.numeric(decimalLongitude),

      # Event date: extract YYYY-MM-DD format
      eventDate = if_else(
        is.na(eventDate),
        NA_character_,
        str_sub(as.character(eventDate), 1L, 10L)
      ),

      # Year: from year column or extract from eventDate
      year = {
        y1 <- suppressWarnings(as.integer(year))
        y2 <- suppressWarnings(as.integer(str_extract(eventDate, "\\d{4}")))
        coalesce(y1, y2)
      },
```

```
      # Bibliographic citation: first 20 chars (avoid minor variations)
      bibliographicCitation = str_sub(
        str_trim(as.character(bibliographicCitation)),
        1, 20
      ),

      # Text fields: trim, lowercase, standardize NAs
      across(
        c(canonicalName, locality, datasetName, catalogNumber,
          occurrenceID, institutionCode),
        ~ {
          x <- str_to_lower(str_trim(as.character(.x)))
          x[x %in% c("", "na", "n a", "n/a")] <- NA_character_
          x
        }
      ),

      # Dataset name: standardize known variations
      datasetName = case_when(
        datasetName %in% c("subpesca-1", "subpesca-2",
                           "fish database correa-boisjoly",
                           "fishgis_metadatabase_ccorrea") ~ datasetName,
        TRUE ~ str_sub(datasetName, 1L, 4L)
      )
    )
}
```

4.3 Apply Standardization

```
table1 <- standardize_for_matching(table1_raw)
table2 <- standardize_for_matching(table2_raw)

cat("Standardization complete.\n")
```

```
## Standardization complete.
```

```
cat("Table1:", nrow(table1), "rows\n")
```

```
## Table1: 911 rows
```

```
cat("Table2:", nrow(table2), "rows\n")
```

```
## Table2: 6760 rows
```

5 Matching Strategy

5.1 Define Matching Keys

Keys are ordered by reliability:

```r
# Core keys: most reliable identifiers
core_keys <- c(
  "datasetName",
  "canonicalName",
  "decimalLatitude",
  "decimalLongitude",
  "bibliographicCitation")

# Extra keys: used to resolve ambiguities
extra_keys <- c(
  "catalogNumber",
  "year",
  "locality",
  "samplingProtocol",
  "individualCount",
  "eventDate",
  "institutionCode",
  "occurrenceID")

all_keys <- c(core_keys, extra_keys)

cat("Matching will use up to", length(all_keys), "keys in hierarchical order\n")
```

```
## Matching will use up to 13 keys in hierarchical order
```

```r
cat("Core keys:", length(core_keys), "\n")
```

```
## Core keys: 5
```

```r
cat("Extra keys:", length(extra_keys), "\n")
```

```
## Extra keys: 8
```

5.2 Strictly Conservative Matching Function

This function adds keys incrementally until a unique match is found. If no unique match is found, no assignment is done, and the match_id field is left blank:

```r
match_tables_strict <- function(table1, table2, core_keys, extra_keys) {

  # Validate inputs
  if (!"id_CC" %in% names(table2)) {
    stop("table2 must contain unique identifier 'id_CC'")
  }

  if (any(duplicated(table2$id_CC))) {
    stop("id_CC in table2 must be unique")
  }

  # Initialize result
  result <- table1 %>%
    mutate(
      row_id = row_number(),
```

```
    match_id = NA_integer_,
    match_keys_used = NA_character_,
    n_keys_used = NA_integer_,
    n_candidates_final = NA_integer_,
    match_status = NA_character_
  )

all_keys <- c(core_keys, extra_keys)
n_core <- length(core_keys)

# Process each row of table1
for (i in seq_len(nrow(table1))) {

  row_i <- table1[i, , drop = FALSE]

  # Start with ALL rows from table2 (no tracking of matched rows)
  candidates <- table2

  keys_used <- character(0)
  keys_available <- character(0)

  # Identify which keys have non-NA values in this table1 row
  for (key in all_keys) {
    if (key %in% names(row_i) && !is.na(row_i[[key]])) {
      keys_available <- c(keys_available, key)
    }
  }

  # Incrementally add keys until unique match or keys exhausted
  for (key in keys_available) {

    val <- row_i[[key]]

    # Filter candidates: exact match on this key, non-NA in table2
    candidates <- candidates %>%
      filter(!is.na(.data[[key]]), .data[[key]] == val)

    keys_used <- c(keys_used, key)

    # Stop if no candidates remain
    if (nrow(candidates) == 0) {
      result$match_status[i] <- "no_match"
      result$n_candidates_final[i] <- 0
      break
    }

    # Stop if unique match achieved
    if (nrow(candidates) == 1) {
      result$match_id[i] <- candidates$id_CC[1]
      result$match_keys_used[i] <- paste(keys_used, collapse = ";")
      result$n_keys_used[i] <- length(keys_used)
      result$n_candidates_final[i] <- 1
      result$match_status[i] <- "matched"
      break
    }
  }
```

```
    # If we exhausted all keys without achieving uniqueness
    if (is.na(result$match_status[i]) && nrow(candidates) > 1) {
      result$match_status[i] <- "ambiguous"
      result$n_candidates_final[i] <- nrow(candidates)
      result$match_keys_used[i] <- paste(keys_used, collapse = ";")
      result$n_keys_used[i] <- length(keys_used)
    }

    # If no keys were available for this row
    if (length(keys_available) == 0) {
      result$match_status[i] <- "no_keys_available"
      result$n_candidates_final[i] <- nrow(table2)
    }
  }

  # Classify match quality based on which keys were used
  result <- result %>%
    mutate(
      n_core_keys_used = if_else(
        !is.na(match_keys_used),
        sapply(strsplit(match_keys_used, ";"), function(keys) {
          sum(keys %in% core_keys)
        }),
        NA_integer_
      ),
      match_quality = case_when(
        match_status != "matched" ~ match_status,
        n_core_keys_used == n_core ~ "all_core_firlds",
        n_core_keys_used >= 4 ~ "4plus_core",
        n_core_keys_used == 3 ~ "3_core",
        n_core_keys_used == 2 ~ "2_core",
        n_core_keys_used <= 1 ~ "1_core",
        TRUE ~ "matched"
      )
    )

  result %>% select(-row_id)
}
```

5.3 Execute Matching

```
cat("Starting strictly conservative matching...\n")
```

```
## Starting strictly conservative matching...
```

```
cat("Each table1 row will be matched independently against ALL table2 rows.\n\n")
```

```
## Each table1 row will be matched independently against ALL table2 rows.
```

```
table1_matched <- match_tables_strict(
  table1 = table1,
  table2 = table2,
```

```
  core_keys = core_keys,
  extra_keys = extra_keys
)

cat("Matching complete.\n")
```

```
## Matching complete.
```

6 Results and Diagnostics

6.1 Overall Matching Success

```
summary_stats <- table1_matched %>%
  summarise(
    total_rows = n(),
    matched = sum(match_status == "matched", na.rm = TRUE),
    ambiguous = sum(match_status == "ambiguous", na.rm = TRUE),
    no_match = sum(match_status == "no_match", na.rm = TRUE),
    no_keys = sum(match_status == "no_keys_available", na.rm = TRUE),
    match_rate = round(100 * matched / total_rows, 1)
  )

kable(summary_stats, caption = "Overall Matching Results")
```

Table 1: Overall Matching Results

total_rows	matched	ambiguous	no_match	no_keys	match_rate
911	878	0	33	0	96.4

6.2 Match Status Distribution

```
status_summary <- table1_matched %>%
  count(match_status) %>%
  mutate(
    percentage = round(100 * n / sum(n), 1)
  ) %>%
  arrange(desc(n))

kable(status_summary, caption = "Match Status Breakdown")
```

Table 2: Match Status Breakdown

match_status	n	percentage
matched	878	96.4
no_match	33	3.6

6.3 Match Quality for Successfully Matched Rows

```
quality_summary <- table1_matched %>%
  filter(match_status == "matched") %>%
  count(match_quality) %>%
  mutate(
    percentage = round(100 * n / sum(n), 1)
  ) %>%
  arrange(desc(n))

kable(quality_summary, caption = "Match Quality Breakdown (Matched Rows Only)")
```

Table 3: Match Quality Breakdown (Matched Rows Only)

match_quality	n	percentage
3_core	612	69.7
all_core_firlds	208	23.7
4plus_core	42	4.8
2_core	16	1.8

```
table1_matched %>%
  filter(match_status == "matched") %>%
  count(match_quality) %>%
  mutate(match_quality = forcats::fct_reorder(match_quality, n)) %>%
  ggplot(aes(x = match_quality, y = n)) +
  geom_col() +
  geom_text(aes(label = n), hjust = -0.2) +
  coord_flip() +
  scale_fill_brewer(palette = "Set3") +
  labs(
    title = "Match Quality Distribution",
    subtitle = "For successfully matched rows only",
    x = "Quality Category",
    y = "Number of Matches"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```

6.4 Keys Required for Matching

```
keys_summary <- table1_matched %>%
  filter(match_status == "matched") %>%
  count(n_keys_used, n_core_keys_used) %>%
  arrange(n_keys_used, desc(n_core_keys_used))

kable(keys_summary, caption = "Number of Keys Required to Achieve Uniqueness")
```

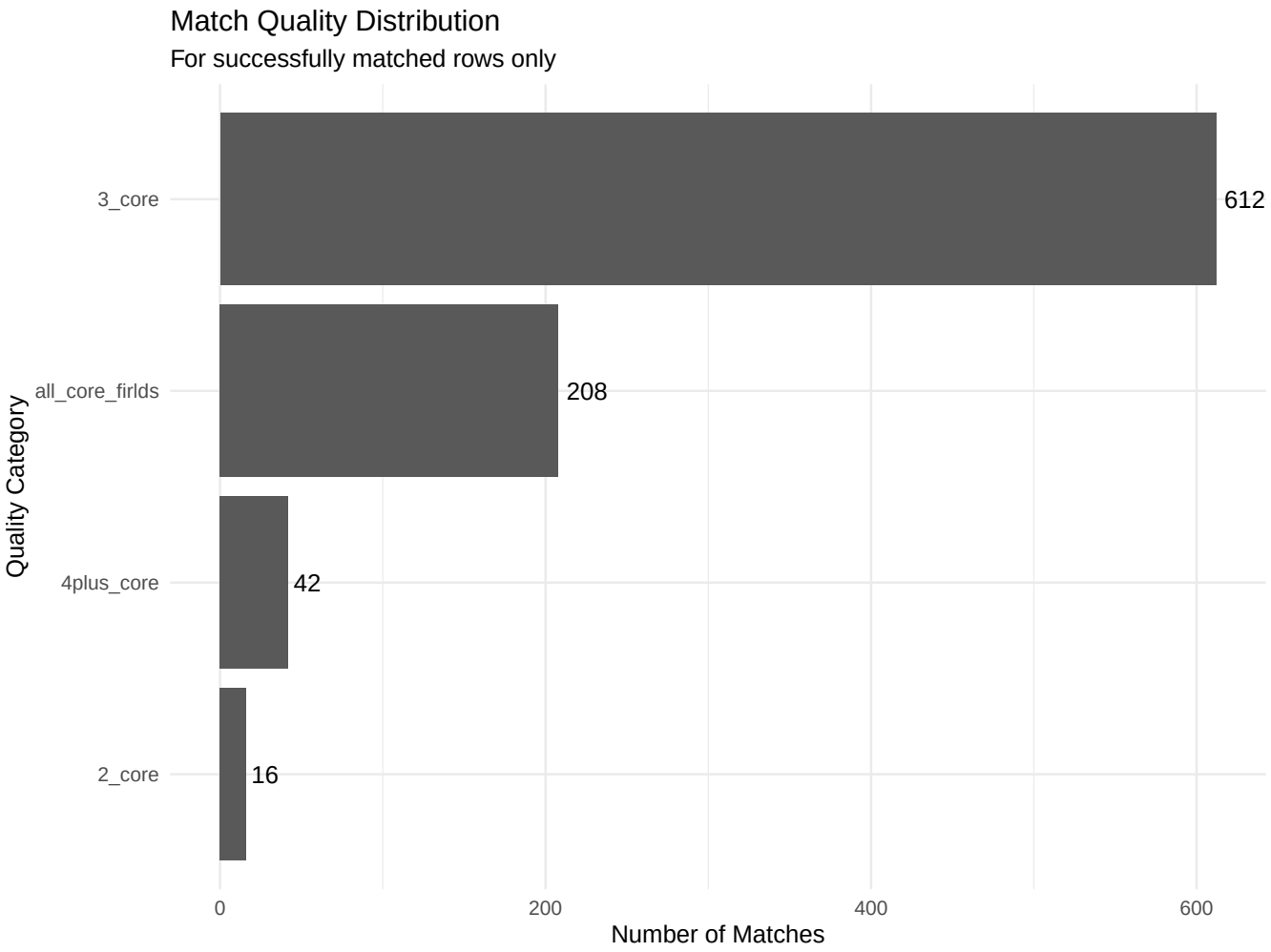


Figure 1: Distribution of Match Quality

Table 4: Number of Keys Required to Achieve Uniqueness

n_keys_used	n_core_keys_used	n
2	2	16
3	3	612
4	4	38
5	5	146
5	4	2
6	5	53
7	4	2
8	5	2
9	5	2
10	5	5

6.5 Specific Key Combinations

Top 20 most common key combinations used:

```
key_combos <- table1_matched %>%
  filter(match_status == "matched") %>%
  count(match_keys_used, sort = TRUE) %>%
  head(20)

kable(key_combos, caption = "Most Common Key Combinations")
```

Table 5: Most Common Key Combinations

match_keys_used	n
datasetName;canonicalName;decimalLatitude	612
datasetName;canonicalName;decimalLatitude;decimalLongitude;bibliographicCitation	146
datasetName;canonicalName;decimalLatitude;decimalLongitude;bibliographicCitation;catalogNumber	53
datasetName;canonicalName;decimalLatitude;decimalLongitude	38
datasetName;canonicalName	15
datasetName;canonicalName;decimalLatitude;decimalLongitude;bibliographicCitation;year;locality;samplingProtocol;individual	5
datasetName;canonicalName;decimalLatitude;decimalLongitude;bibliographicCitation;year;locality;samplingProtocol	2
datasetName;canonicalName;decimalLatitude;decimalLongitude;bibliographicCitation;year;locality;samplingProtocol;individual	2
datasetName;canonicalName;decimalLatitude;decimalLongitude;catalogNumber;year;eventDate	2
datasetName;canonicalName;decimalLatitude;decimalLongitude;catalogNumber	1
datasetName;canonicalName;decimalLatitude;decimalLongitude;year	1
datasetName;decimalLatitude	1

6.6 Uniqueness Validation

Critical check: verify each match_id appears only once:

```
duplicate_matches <- table1_matched %>%
  filter(match_status == "matched") %>%
  count(match_id, name = "frequency") %>%
  filter(frequency > 1)

if (nrow(duplicate_matches) > 0) {
  cat(" ERROR:", nrow(duplicate_matches), "table2 rows matched multiple times!\n")
```

```
  kable(head(duplicate_matches, 20), caption = "Duplicate Matches (CRITICAL ERROR)")
} else {
  cat(" SUCCESS: All matches are unique (one-to-one)\n")
  cat("Each match_id appears exactly once.\n")
}
```

```
##  ERROR: 2 table2 rows matched multiple times!
```

Table 6: Duplicate Matches (CRITICAL ERROR)

match_id	frequency
11653	2
12426	2

```
table1_matched %>%
  filter(match_status == "matched") %>%
  count(match_id, name = "freq") %>%
  ggplot(aes(x = freq)) +
  geom_histogram(binwidth = 1, color = "black", fill = "steelblue") +
  geom_vline(xintercept = 1, color = "red", linetype = "dashed", size = 1) +
  annotate("text", x = 1.5, y = Inf, label = "All should be at 1",
           vjust = 2, color = "red", fontface = "bold") +
  labs(
    title = "Distribution of match_id Frequency",
    subtitle = "Perfect one-to-one matching requires all frequencies = 1",
    x = "Times each match_id appears",
    y = "Count"
  ) +
  theme_minimal()
```

6.7 Ambiguous Matches Analysis

Rows that had multiple candidates even after using all available keys:

```
if (sum(table1_matched$match_status == "ambiguous", na.rm = TRUE) > 0) {

  ambiguous_summary <- table1_matched %>%
    filter(match_status == "ambiguous") %>%
    group_by(n_candidates_final) %>%
    summarise(
      count = n(),
      avg_keys_used = round(mean(n_keys_used, na.rm = TRUE), 1)
    ) %>%
    arrange(desc(count))

  kable(ambiguous_summary, caption = "Ambiguous Matches: Number of Final Candidates")

  # Show examples of ambiguous matches
  ambiguous_examples <- table1_matched %>%
    filter(match_status == "ambiguous") %>%
    select(
      id_CC, datasetName, canonicalName, year,
```

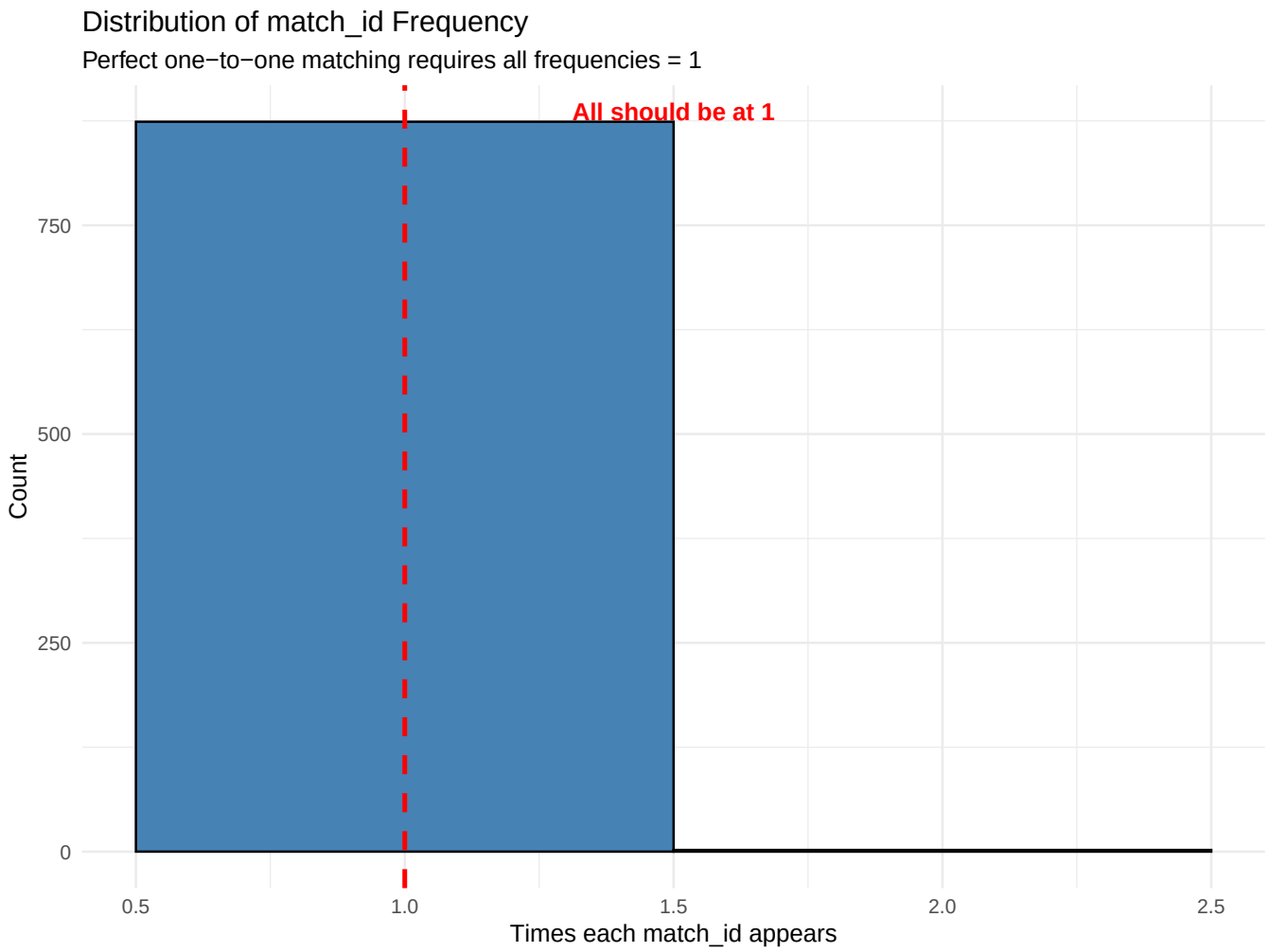


Figure 2: Match ID Frequency Distribution

```
      n_candidates_final, n_keys_used, match_keys_used
    ) %>%
    head(10)

  kable(ambiguous_examples, caption = "Examples of Ambiguous Matches")

} else {
  cat("No ambiguous matches found.\n")
}
```

```
## No ambiguous matches found.
```

6.8 No Match Analysis

Rows where no table2 row matched the criteria:

```
if (sum(table1_matched$match_status == "no_match", na.rm = TRUE) > 0) {

  no_match_summary <- table1_matched %>%
    filter(match_status == "no_match") %>%
    count(datasetName) %>%
    arrange(desc(n))

  kable(no_match_summary, caption = "No Match Cases by Dataset")

} else {
  cat("All rows either matched or were ambiguous.\n")
}
```

Table 7: No Match Cases by Dataset

datasetName	n
gbif	25
mma2	4
agui	2
fishgis_metadatabase_ccorrea	1
idig	1

6.9 Unmatched Rows Requiring Review

Exclude expected unmatched categories:

```
unmatched_review <- table1_matched %>%
  filter(
     match_status != "matched",
        ) %>%
  arrange(match_status, datasetName, canonicalName, year)

cat("Unmatched rows requiring manual review:", nrow(unmatched_review), "\n\n")
```

```
## Unmatched rows requiring manual review: 33
```

```r
unmatched_by_status <- unmatched_review %>%
  count(match_status, datasetName) %>%
  arrange(match_status, desc(n))

kable(unmatched_by_status, caption = "Unmatched Rows by Status and Dataset")
```

Table 8: Unmatched Rows by Status and Dataset

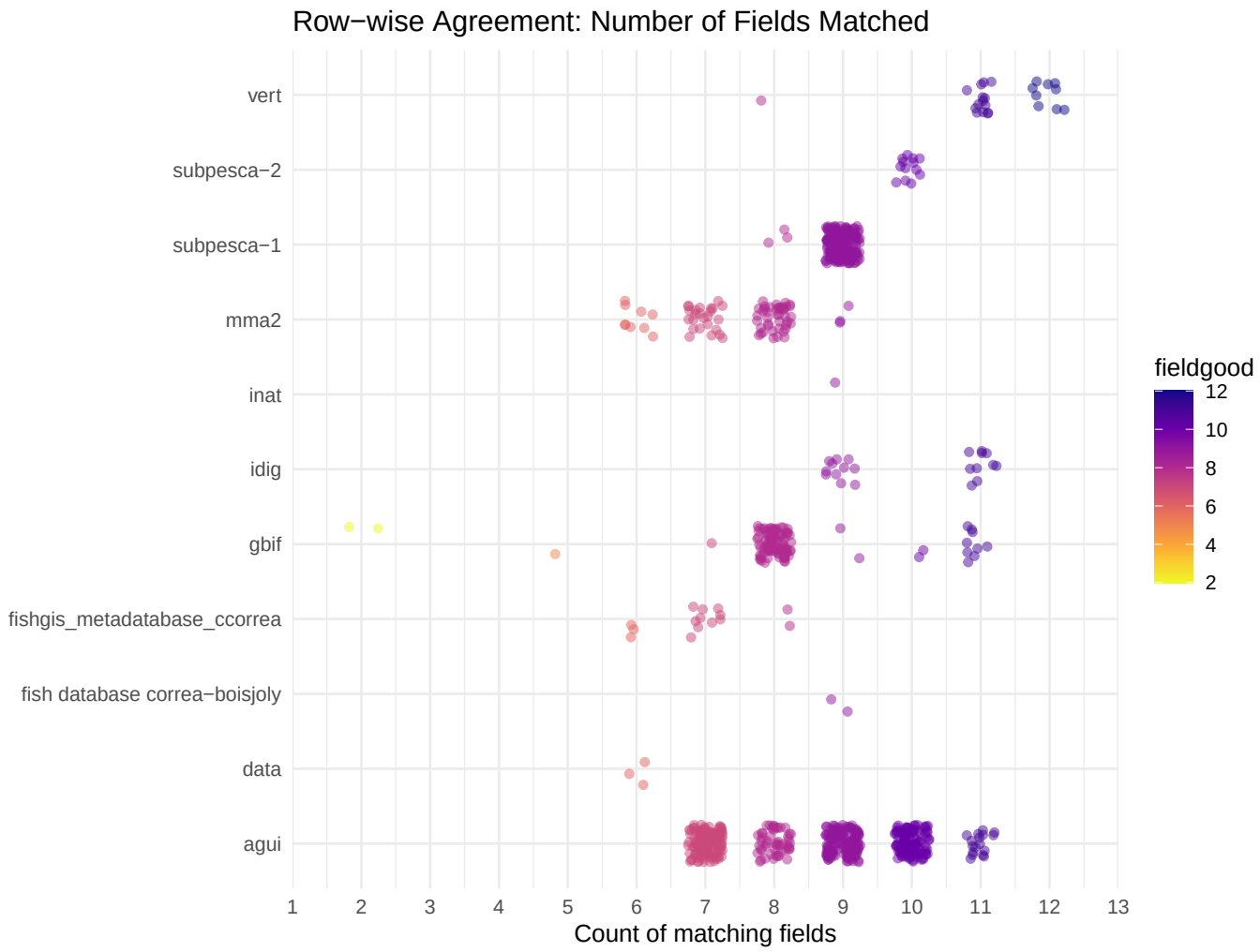
match_status	datasetName	n
no_match	gbif	25
no_match	mma2	4
no_match	agui	2
no_match	fishgis_metadatabase_ccorrea	1
no_match	idig	1

6.10 Matched, rowwise field-by-Field Comparison

For every matched row, comare pairs of corresponding fields, and anayze the frequencies of matching and miss-matching filds, and the log ratio of matches to miss-matches.

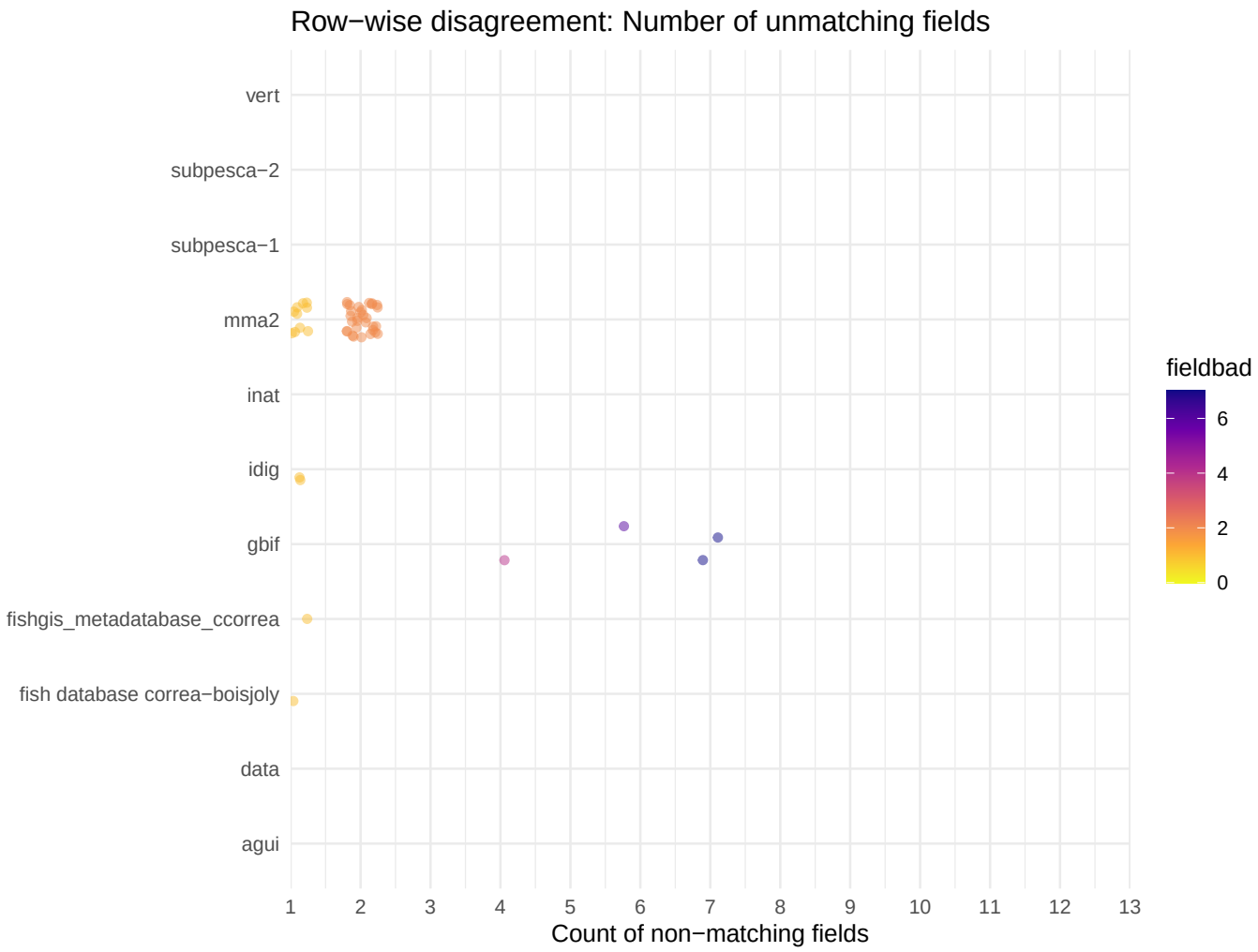
```r
if (sum(table1_matched$match_status == "matched", na.rm = TRUE) > 0) {

# matched subsets
tb1 <- table1_matched %>%
  filter(match_status == "matched") %>%
  select(
    match_id,
    any_of(setdiff(colnames(table2), c("id_CC", starts_with("."))))
  ) %>%
  arrange(match_id)

tb2 <- table2 %>%
  filter(id_CC %in% tb1$match_id) %>%
  arrange(id_CC)

tb2 <- tb2[match(tb1$match_id, tb2$id_CC), ]

comparison_cols <- setdiff(intersect(names(tb1), names(tb2)), "match_id")

diff.strict <- tb1[, comparison_cols] == tb2[, comparison_cols]

tmp <- table1_matched %>%
  filter(match_status == "matched") %>%
  select(
    id_CC_old = id_CC,
    match_id,
    match_keys_used,
    dataset = datasetName
  ) %>%
  arrange(match_id)

# logical match matrix -> data frame
diff.strict <- as.data.frame(diff.strict)
```

```r
# add metadata for each matched row
diff.strict <- bind_cols(
  tmp[!is.na(tmp$match_id), ],  # keep only matched rows in tmp
  diff.strict
)

# compute number of matching fields row-wise
diff.strict <- diff.strict %>%
  mutate(fieldgood = rowSums(across(!c(id_CC_old, match_id, match_keys_used, dataset)), na.rm = TRUE))

# plot
ggplot(diff.strict, aes(x = dataset, y = fieldgood)) +
  geom_jitter(aes(color = fieldgood),
              height = 0.25, width = 0.25, alpha = 0.5) +
  coord_flip() +
  scale_y_continuous(breaks = seq(1, length(comparison_cols)),
                     limits= c(1, length(comparison_cols)),
                     expand = FALSE) +
  scale_color_viridis_c(option = "plasma", direction = -1) +
  labs(
    title = "Row-wise Agreement: Number of Fields Matched",
    x = NULL,
    y = "Count of matching fields"
  ) +
  theme_minimal()

} else {
  cat("No matched rows to compare.\n")
}
```




```r
 if (sum(table1_matched$match_status == "matched", na.rm = TRUE) > 0) {

# matched subsets
tb1 <- table1_matched %>%
  filter(match_status == "matched") %>%
  select(
    match_id,
    any_of(setdiff(colnames(table2), c("id_CC", starts_with("."))))
  ) %>%
  arrange(match_id)

tb2 <- table2 %>%
  filter(id_CC %in% tb1$match_id) %>%
  arrange(id_CC)

tb2 <- tb2[match(tb1$match_id, tb2$id_CC), ]

comparison_cols <- setdiff(intersect(names(tb1), names(tb2)), "match_id")

# COUNT DIFFERENCES THIS TIME
diff.strict2 <- tb1[, comparison_cols] != tb2[, comparison_cols]

tmp <- table1_matched %>%
  filter(match_status == "matched") %>%
```

```r
  select(
    id_CC_old = id_CC,
    match_id,
    match_keys_used,
    dataset = datasetName
  ) %>%
  arrange(match_id)

# logical match matrix -> data frame
diff.strict2 <- as.data.frame(diff.strict2)

# add metadata for each matched row
diff.strict2 <- bind_cols(
  tmp[!is.na(tmp$match_id), ],  # keep only matched rows in tmp
  diff.strict2
)

# compute number of matching fields row-wise
diff.strict.falses <- diff.strict2 %>%
  mutate(fieldbad = rowSums(across(!c(id_CC_old, match_id, match_keys_used, dataset)), na.rm = TRUE))

# plot
ggplot(diff.strict.falses, aes(x = dataset, y = fieldbad)) +
  geom_jitter(aes(color = fieldbad),
              height = 0.25, width = 0.25, alpha = 0.5) +
  coord_flip() +
  scale_y_continuous(breaks = seq(1, length(comparison_cols)),
                     limits= c(1, length(comparison_cols)),
                     expand = FALSE) +
  scale_color_viridis_c(option = "plasma", direction = -1) +
  labs(
    title = "Row-wise disagreement: Number of unmatching fields",
    x = NULL,
    y = "Count of non-matching fields"
  ) +
  theme_minimal()

} else {
  cat("No matched rows to compare.\n")
}
```




```
colnames(diff.strict2)
```

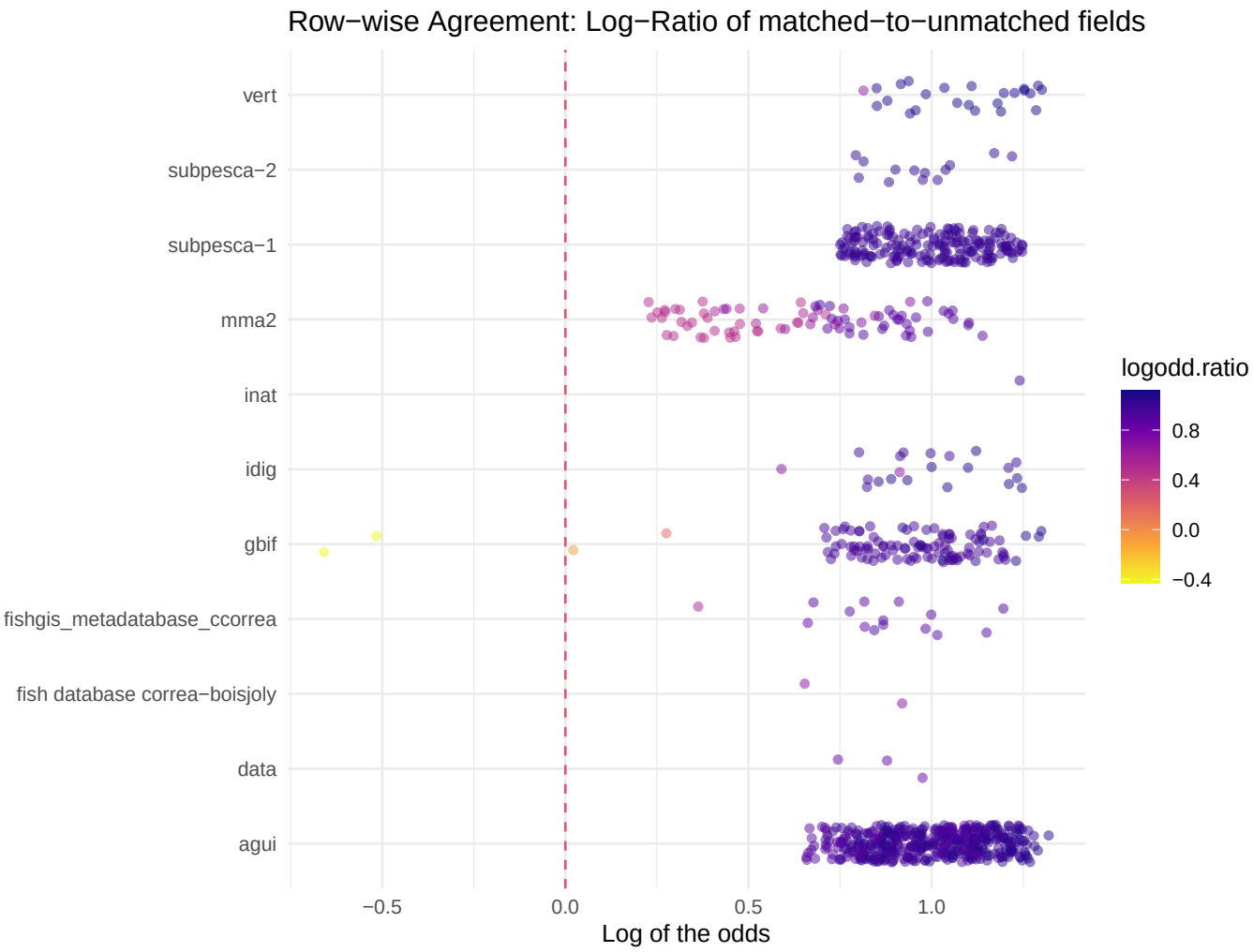
```
##  [1] "id_CC_old"             "match_id"              "match_keys_used"
##  [4] "dataset"               "decimalLatitude"       "decimalLongitude"
##  [7] "canonicalName"         "year"                  "eventDate"
## [10] "locality"              "samplingProtocol"      "individualCount"
## [13] "catalogNumber"         "datasetName"           "bibliographicCitation"
## [16] "institutionCode"       "occurrenceID"
```

```
good.and.bad.matches <-  diff.strict %>% full_join(diff.strict.falses, by = "id_CC_old", keep = T) %>%
  select(dataset.x, id_CC_old.x, match_id.x, fieldgood, fieldbad) %>%
  mutate(logodd.ratio = log10((fieldgood + 1) / (fieldbad + 1)),
         precent.good = 100 * (fieldgood /length(comparison_cols)),
         percent.bad = 100 *  (fieldbad /length(comparison_cols)))

good.and.bad.matches %>% filter(fieldbad >= 3) %>%
  kable(caption = "Matched rows with 3 or more missmatches.")
```

Table 9: Matched rows with 3 or more missmatches.

dataset.x	id_CC_old.x	match_id.x	fieldgood	fieldbad	logodd.ratio	precent.good	percent.bad
gbif	549	451	7	4	0.2041200	53.84615	30.76923
gbif	12726	11653	5	6	-0.0669468	38.46154	46.15385
gbif	7611	12426	2	7	-0.4259687	15.38462	53.84615
gbif	12328	12426	2	7	-0.4259687	15.38462	53.84615

```
good.and.bad.matches %>%
ggplot(aes(x = dataset.x, y = logodd.ratio)) +
  geom_jitter(aes(color = logodd.ratio),
              height = 0.25, width = 0.25, alpha = 0.5) +
  coord_flip() +
  # scale_y_continuous(breaks = seq(1, length(comparison_cols)),
  #                    limits= c(1, length(comparison_cols)),
  #                    expand = FALSE) +
  scale_color_viridis_c(option = "plasma", direction = -1) +
  geom_hline(yintercept = 0, lty=2, col=2)+
  labs(
    title = "Row-wise Agreement: Log-Ratio of matched-to-unmatched fields",
    x = NULL,
    y = "Log of the odds"
  ) +
  theme_minimal()
```



6.11 Matched, overall field-by-Field Comparison

For successfully matched rows, compare original values:

```
if (sum(table1_matched$match_status == "matched", na.rm = TRUE) > 0) {

 # Prepare matched subsets
 tb1 <- table1_matched %>%
   filter(match_status == "matched") %>%
   select(
     match_id,
     any_of(setdiff(colnames(table2), c("id_CC", starts_with("."))))
   ) %>%
   arrange(match_id)

 tb2 <- table2 %>%
   filter(id_CC %in% tb1$match_id) %>%
   arrange(id_CC)

 # Ensure same order
 tb2 <- tb2[match(tb1$match_id, tb2$id_CC), ]

 # Compare field by field
```

```r
  comparison_cols <- intersect(names(tb1), names(tb2))
  comparison_cols <- setdiff(comparison_cols, "match_id")

  agreement <- sapply(comparison_cols, function(col) {
    sum(tb1[[col]] == tb2[[col]], na.rm = TRUE)
  })

  total <- nrow(tb1)

  agreement_summary <- data.frame(
    field = comparison_cols,
    agreements = agreement,
    total = total,
    agreement_rate = round(100 * agreement / total, 1)
  ) %>%
    arrange(desc(agreement_rate))

  kable(agreement_summary, caption = "Field Agreement Rates for Matched Rows")

} else {
  cat("No matched rows to compare.\n")
}
```

Table 10: Field Agreement Rates for Matched Rows

	field	agreements	total	agreement_rate
datasetName	datasetName	878	878	100.0
canonicalName	canonicalName	877	878	99.9
decimalLatitude	decimalLatitude	876	878	99.8
decimalLongitude	decimalLongitude	875	878	99.7
bibliographicCitation	bibliographicCitation	703	878	80.1
catalogNumber	catalogNumber	668	878	76.1
year	year	658	878	74.9
locality	locality	594	878	67.7
samplingProtocol	samplingProtocol	524	878	59.7
individualCount	individualCount	447	878	50.9
eventDate	eventDate	176	878	20.0
occurrenceID	occurrenceID	153	878	17.4
institutionCode	institutionCode	152	878	17.3

```r
if (sum(table1_matched$match_status == "matched", na.rm = TRUE) > 0) {
  agreement_summary %>%
    mutate(field = forcats::fct_reorder(field, agreement_rate)) %>%
    ggplot(aes(x = field, y = agreement_rate, fill = agreement_rate)) +
    geom_col() +
    geom_hline(yintercept = 100, linetype = "dashed", color = "red") +
    coord_flip() +
    scale_fill_gradient(low = "orange", high = "darkgreen") +
    labs(
      title = "Field Agreement Rates",
      subtitle = "Comparison between matched table1 and table2 rows",
      x = "Field",
      y = "Agreement Rate (%)",
      fill = "Agreement %"
```

```
    ) +
    theme_minimal()
}
```

```
if (sum(table1_matched$match_status == "matched", na.rm = TRUE) > 0) {

  # Prepare matched subsets, keep datasetName
  tb1 <- table1_matched %>%
    filter(match_status == "matched") %>%
    select(
      datasetName,        # keep for stratification
      match_id,
      any_of(setdiff(colnames(table2), c("id_CC", starts_with("."))))
    ) %>%
    arrange(match_id)

  tb2 <- table2 %>%
    filter(id_CC %in% tb1$match_id) %>%
    arrange(id_CC)

  # Ensure same order
  tb2 <- tb2[match(tb1$match_id, tb2$id_CC), ]

  # Columns to compare
  comparison_cols <- intersect(names(tb1), names(tb2))
  comparison_cols <- setdiff(comparison_cols, c("match_id", "datasetName"))

  # Logical matrix of equalities for each field
  equal_df <- purrr::map_dfc(
    comparison_cols,
    ~ tb1[[.x]] == tb2[[.x]]
  )
  names(equal_df) <- comparison_cols

  # Long format: one row per (datasetName, field, row-pair)
  agreement_summary <- bind_cols(
    datasetName = tb1$datasetName,
    equal_df
  ) %>%
    pivot_longer(
      cols = -datasetName,
      names_to = "field",
      values_to = "equal"
    ) %>%
    group_by(datasetName, field) %>%
    summarise(
      agreements     = sum(equal, na.rm = TRUE),
      total          = sum(!is.na(equal)),
      agreement_rate = round(100 * agreements / total, 1),
      .groups        = "drop"
    ) %>%
    arrange(datasetName, desc(agreement_rate))

  kable(agreement_summary, caption = "Field Agreement Rates for Matched Rows, by datasetName")

} else {
```

```
  cat("No matched rows to compare.\n")
}
```

Table 11: Field Agreement Rates for Matched Rows, by datasetName

datasetName	field	agreements	total	agreement_rate
agui	bibliographicCitation	429	429	100.0
agui	canonicalName	429	429	100.0
agui	catalogNumber	429	429	100.0
agui	decimalLatitude	429	429	100.0
agui	decimalLongitude	429	429	100.0
agui	eventDate	25	25	100.0
agui	individualCount	140	140	100.0
agui	locality	429	429	100.0
agui	samplingProtocol	288	288	100.0
agui	year	251	251	100.0
agui	institutionCode	0	0	NaN
agui	occurrenceID	0	0	NaN
data	canonicalName	3	3	100.0
data	decimalLatitude	3	3	100.0
data	decimalLongitude	3	3	100.0
data	institutionCode	3	3	100.0
data	occurrenceID	3	3	100.0
data	bibliographicCitation	0	0	NaN
data	catalogNumber	0	0	NaN
data	eventDate	0	0	NaN
data	individualCount	0	0	NaN
data	locality	0	0	NaN
data	samplingProtocol	0	0	NaN
data	year	0	0	NaN
fish database correa-boisjoly	canonicalName	2	2	100.0
fish database correa-boisjoly	decimalLatitude	2	2	100.0
fish database correa-boisjoly	decimalLongitude	2	2	100.0
fish database correa-boisjoly	eventDate	2	2	100.0
fish database correa-boisjoly	individualCount	2	2	100.0
fish database correa-boisjoly	locality	2	2	100.0
fish database correa-boisjoly	samplingProtocol	2	2	100.0
fish database correa-boisjoly	year	2	2	100.0
fish database correa-boisjoly	bibliographicCitation	0	2	0.0
fish database correa-boisjoly	catalogNumber	0	0	NaN
fish database correa-boisjoly	institutionCode	0	0	NaN
fish database correa-boisjoly	occurrenceID	0	0	NaN
fishgis_metadatabase_ccorrea	bibliographicCitation	15	15	100.0
fishgis_metadatabase_ccorrea	canonicalName	14	14	100.0
fishgis_metadatabase_ccorrea	decimalLatitude	15	15	100.0
fishgis_metadatabase_ccorrea	decimalLongitude	15	15	100.0
fishgis_metadatabase_ccorrea	eventDate	2	2	100.0
fishgis_metadatabase_ccorrea	year	14	14	100.0
fishgis_metadatabase_ccorrea	locality	14	15	93.3
fishgis_metadatabase_ccorrea	catalogNumber	0	0	NaN
fishgis_metadatabase_ccorrea	individualCount	0	0	NaN
fishgis_metadatabase_ccorrea	institutionCode	0	0	NaN
fishgis_metadatabase_ccorrea	occurrenceID	0	0	NaN
fishgis_metadatabase_ccorrea	samplingProtocol	0	0	NaN

datasetName	field	agreements	total	agreement_rate
gbif	canonicalName	108	108	100.0
gbif	eventDate	106	106	100.0
gbif	year	106	106	100.0
gbif	decimalLatitude	106	108	98.1
gbif	decimalLongitude	105	108	97.2
gbif	institutionCode	104	107	97.2
gbif	occurrenceID	104	108	96.3
gbif	catalogNumber	13	17	76.5
gbif	locality	12	16	75.0
gbif	individualCount	9	13	69.2
gbif	bibliographicCitation	0	0	NaN
gbif	samplingProtocol	0	0	NaN
idig	canonicalName	21	21	100.0
idig	decimalLatitude	21	21	100.0
idig	decimalLongitude	21	21	100.0
idig	individualCount	17	17	100.0
idig	institutionCode	21	21	100.0
idig	locality	20	20	100.0
idig	occurrenceID	21	21	100.0
idig	samplingProtocol	3	3	100.0
idig	year	12	12	100.0
idig	catalogNumber	20	21	95.2
idig	eventDate	11	12	91.7
idig	bibliographicCitation	0	0	NaN
inat	canonicalName	1	1	100.0
inat	catalogNumber	1	1	100.0
inat	decimalLatitude	1	1	100.0
inat	decimalLongitude	1	1	100.0
inat	eventDate	1	1	100.0
inat	locality	1	1	100.0
inat	occurrenceID	1	1	100.0
inat	year	1	1	100.0
inat	bibliographicCitation	0	0	NaN
inat	individualCount	0	0	NaN
inat	institutionCode	0	0	NaN
inat	samplingProtocol	0	0	NaN
mma2	canonicalName	80	80	100.0
mma2	decimalLatitude	80	80	100.0
mma2	decimalLongitude	80	80	100.0
mma2	individualCount	65	65	100.0
mma2	locality	80	80	100.0
mma2	samplingProtocol	35	35	100.0
mma2	year	57	57	100.0
mma2	bibliographicCitation	40	78	51.3
mma2	eventDate	3	43	7.0
mma2	catalogNumber	0	0	NaN
mma2	institutionCode	0	0	NaN
mma2	occurrenceID	0	0	NaN
subpesca-1	bibliographicCitation	182	182	100.0
subpesca-1	canonicalName	182	182	100.0
subpesca-1	catalogNumber	182	182	100.0
subpesca-1	decimalLatitude	182	182	100.0
subpesca-1	decimalLongitude	182	182	100.0
subpesca-1	individualCount	182	182	100.0
subpesca-1	samplingProtocol	182	182	100.0

datasetName	field	agreements	total	agreement_rate
subpesca-1	year	179	179	100.0
subpesca-1	eventDate	0	0	NaN
subpesca-1	institutionCode	0	0	NaN
subpesca-1	locality	0	0	NaN
subpesca-1	occurrenceID	0	0	NaN
subpesca-2	bibliographicCitation	13	13	100.0
subpesca-2	canonicalName	13	13	100.0
subpesca-2	decimalLatitude	13	13	100.0
subpesca-2	decimalLongitude	13	13	100.0
subpesca-2	eventDate	13	13	100.0
subpesca-2	individualCount	13	13	100.0
subpesca-2	locality	13	13	100.0
subpesca-2	samplingProtocol	13	13	100.0
subpesca-2	year	13	13	100.0
subpesca-2	catalogNumber	0	0	NaN
subpesca-2	institutionCode	0	0	NaN
subpesca-2	occurrenceID	0	0	NaN
vert	bibliographicCitation	24	24	100.0
vert	canonicalName	24	24	100.0
vert	catalogNumber	23	23	100.0
vert	decimalLatitude	24	24	100.0
vert	decimalLongitude	24	24	100.0
vert	eventDate	13	13	100.0
vert	individualCount	19	19	100.0
vert	institutionCode	24	24	100.0
vert	occurrenceID	24	24	100.0
vert	samplingProtocol	1	1	100.0
vert	year	23	23	100.0
vert	locality	23	24	95.8

```r
if (sum(table1_matched$match_status == "matched", na.rm = TRUE) > 0) {
  agreement_summary %>%
    mutate(field = forcats::fct_reorder(field, agreement_rate)) %>%
    ggplot(aes(x = field, y = agreement_rate, fill = agreement_rate)) +
    geom_col() +
    geom_hline(yintercept = 100, linetype = "dashed", color = "red") +
    coord_flip() +
    scale_fill_gradient(low = "orange", high = "darkgreen") +
    facet_wrap(~ datasetName) +
    labs(
      title = "Field Agreement Rates by datasetName",
      subtitle = "Comparison between matched table1 and table2 rows",
      x = "Field",
      y = "Agreement Rate (%)",
      fill = "Agreement %"
    ) +
    theme_minimal()
}
```

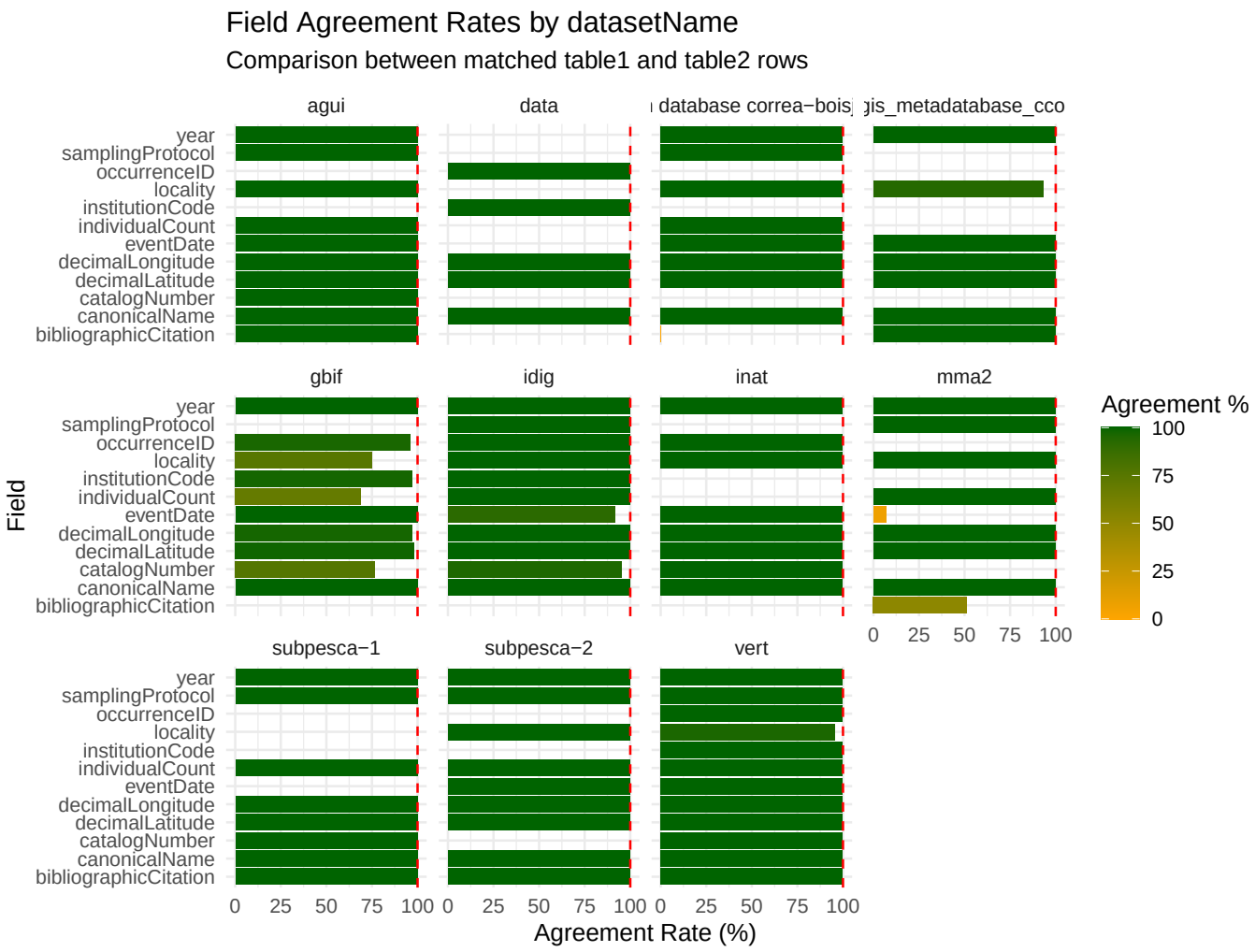


Figure 3: Field Agreement Rates by datasetName

7 Export Results

7.1 Matched Records

```
table1_export <- table1_matched %>%
  # left_join(
  #   table1_raw %>% select(id_CC, starts_with(".")),
  #   by = "id_CC"
  # ) %>%
  select(
    id_CC_original = id_CC,
    match_id,
    match_status,
    match_quality,
    n_keys_used,
    n_core_keys_used,
    n_candidates_final,
    match_keys_used,
    datasetName,
    canonicalName,
    year,
    eventDate,
    locality,
    decimalLatitude,
    decimalLongitude,
    bibliographicCitation,
    catalogNumber,
    occurrenceID,
    institutionCode,
    Duplicate,
    Comment,
    starts_with(".")
  ) %>%
  arrange(as.numeric(id_CC_original)) #%>%
  #arrange(match_status, match_quality, datasetName, canonicalName, year)

write.csv( table1_export,
  "./outputs/Table1_matched_strict_2025-12-10.csv",
  fileEncoding = "UTF-8", row.names = FALSE )

# But see Utility function at the end to import previos annotations, if available.

cat("Exported", nrow(table1_export), "rows to Table1_matched_strict.csv\n")
```

```
## Exported 911 rows to Table1_matched_strict.csv
```

7.2 Good-to-bad match ratio

```
write.csv(
    good.and.bad.matches,
    "./outputs/Table1_good-bad_matches_2025-12-09.csv",
    fileEncoding = "UTF-8",
```

```
    row.names = FALSE
  )
```

7.3 Unmatched Records for Manual Review

```
if (exists("unmatched_review") && nrow(unmatched_review) > 0) {
  write.csv(
    unmatched_review,
    "./outputs/joining_tables/Table1_unmatched_review.csv",
    fileEncoding = "UTF-8",
    row.names = FALSE
  )

  cat("Exported", nrow(unmatched_review), "unmatched rows for review\n")
}
```

7.4 Ambiguous Matches for Investigation

```
ambiguous_export <- table1_matched %>%
  filter(match_status == "ambiguous") %>%
  select(
    id_CC,
    n_candidates_final,
    n_keys_used,
    match_keys_used,
    datasetName,
    canonicalName,
    year,
    locality,
    decimalLatitude,
    decimalLongitude,
    bibliographicCitation,
    catalogNumber,
    occurrenceID,
    institutionCode
  ) %>%
  arrange(desc(n_candidates_final), datasetName)

if (nrow(ambiguous_export) > 0) {
  write.csv(
    ambiguous_export,
    "./outputs/joining_tables/Table1_ambiguous_matches.csv",
    fileEncoding = "UTF-8",
    row.names = FALSE
  )

  cat("Exported", nrow(ambiguous_export), "ambiguous matches for investigation\n")
}
```

8 Summary Statistics Table

```
summary_table <- data.frame(
  Metric = c(
    "Total table1 rows",
    "Successfully matched",
    "Ambiguous (multiple candidates)",
    "No match found",
    "No keys available",
    "Match rate (%)",
    "One-to-one constraint satisfied"
  ),
  Value = c(
    nrow(table1_matched),
    sum(table1_matched$match_status == "matched", na.rm = TRUE),
    sum(table1_matched$match_status == "ambiguous", na.rm = TRUE),
    sum(table1_matched$match_status == "no_match", na.rm = TRUE),
    sum(table1_matched$match_status == "no_keys_available", na.rm = TRUE),
    round(100 * sum(table1_matched$match_status == "matched", na.rm = TRUE) / nrow(table1_matched), 1),
    ifelse(nrow(duplicate_matches) == 0, "Yes ", "No ")
  )
)

kable(summary_table, caption = "Final Summary Statistics")
```

Table 12: Final Summary Statistics

Metric	Value
Total table1 rows	911
Successfully matched	878
Ambiguous (multiple candidates)	0
No match found	33
No keys available	0
Match rate (%)	96.4
One-to-one constraint satisfied	No

9 Final remarks

- **Total processed:** 911 rows
- **Successfully matched:** 878 rows
- **Match rate:** 96.4%
- **One-to-one verified:** No

This strictly conservative matching strategy ensures maximum reliability by:

1. **No tracking of matched table2 rows**: Each table1 row is evaluated independently against the entire table2
2. **Incremental key addition**: Keys are added one-by-one until uniqueness is achieved

3. **Explicit ambiguity detection**: Rows with multiple candidates are flagged as ambiguous

4. **Complete transparency**: All key combinations used are recorded for validation

9.1 Final Outputs

The automated record-matching workflow successfully identified most correct links between the master consolidated dataset and the AtlasIctio corrections table. Records flagged as unmatched, low-confidence, or ambiguous were manually checked to ensure traceability and accuracy.

All verification results are now compiled in `Table1_matched_strict_reviewed_2025-12-09.csv`. This file establishes the mapping between:

- `dataset_before_Remocion_manual_2025-12-03.rds` (consolidated dataset prior to manual review)
- `AtlasIctio_COMMENTS_2025_12.csv` (corrected annotations table)

The reviewed file includes the following fields required for updating identifiers:

- `id_CC_original`: ID of the consolidated dataset row before manual revision
- `match_id`: ID in the deduplicated dataset (`X.filtered.dedup_2025-12-02.csv`) determined by the matching workflow
- `correcting_match`: manually validated match (integer if corrected match_id, or explanatory note if text)

9.2 Next Steps

- Integrate the reviewed match table into the main data processing workflow
- Before joining to the main table, replace outdated match_id values where manual correction exists

10 Session Information

```
sessionInfo()
```

```
## R version 4.4.1 (2024-06-14 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26100)
##
## Matrix products: default
##
##
## locale:
## [1] LC_COLLATE=English_Canada.utf8  LC_CTYPE=English_Canada.utf8
## [3] LC_MONETARY=English_Canada.utf8 LC_NUMERIC=C
## [5] LC_TIME=English_Canada.utf8
##
## time zone: America/Santiago
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
```

```
##
## other attached packages:
## [1] tidyr_1.3.1   knitr_1.50    ggplot2_4.0.1 stringr_1.6.0 dplyr_1.1.4
##
## loaded via a namespace (and not attached):
##  [1] vctrs_0.6.5        cli_3.6.5          rlang_1.1.6        xfun_0.54
##  [5] forcats_1.0.1      stringi_1.8.7      purrr_1.2.0        generics_0.1.4
##  [9] S7_0.2.1           labeling_0.4.3     glue_1.8.0         htmltools_0.5.8.1
## [13] tinytex_0.58       scales_1.4.0       rmarkdown_2.30     grid_4.4.1
## [17] evaluate_1.0.5     tibble_3.3.0       fastmap_1.2.0      yaml_2.3.10
## [21] lifecycle_1.0.4    compiler_4.4.1     RColorBrewer_1.1-3 pkgconfig_2.0.3
## [25] rstudioapi_0.17.1  farver_2.1.2       digest_0.6.39      viridisLite_0.4.2
## [29] R6_2.6.1           tidyselect_1.2.1   pillar_1.11.1      magrittr_2.0.4
## [33] withr_3.0.2        tools_4.4.1        gtable_0.3.6
```

11 Utility function to compare assignments and import manually revised annotation to new matching tables.

```
# ======================================================================
# import_annotations() - Import Reviewed Annotations by Composite Key
# ======================================================================
#
# This function helps migrate manually reviewed annotations (e.g.,
# match corrections) from an older version of a dataset (`table_old`)
# to a newly generated version (`table_new`).
#
# It matches rows based on a customizable set of key columns
# (default: id_CC_original + match_id) and imports the annotation
# column (default: "correcting_match") into the new table.
#
# The function also adds diagnostic columns to the output for easy
# filtering and manual review. These indicate whether each row was
# previously reviewed, whether the annotation was successfully
# imported, and whether the original ID or match has changed.
#
# Output:
# - `updated_table`: Copy of the new table with annotations and diagnostics
# - `diagnostics`: Summary stats and rows needing review
#
# Ideal for use in iterative data matching, de-duplication, and
# validation workflows.


# Load utility functiuon to compare two versions of matching tables, the older with comments in correcting

source("./annotated-new_output_comparisson/compare_and_import_annotations.R")

# Load the (older) annotated matched table

table1.annotated <- read.csv("./outputs/Table1_matched_strict_reviewed_2025-12-06.csv",
                encoding = "UTF-8", sep = ",", colClasses = "character",
                na.strings = c("", "NA"))
```

```r
# Load the new version lacking manual annotations, or use the object generated in this script.
# Run the comparisson function and import the correcting_match filed into the new table only if the compos

  result <- import_annotations(
    table_old = table1.annotated,
    table_new = table1_export,
    key_columns = c("id_CC_original", "match_id"),
    annotation_column = "correcting_match"
  )
```

```
##
## ========================================
## IMPORTING ANNOTATIONS
## ========================================
##
## === SUMMARY ===
##                                     metric count
##              Rows with imported annotations    59
##  Rows needing manual review (no annotation)   852
##                      Total rows in new table   911
## WARNING: 852 rows need manual review
## Done!
```

```r
# Access results

# Main updated table
updated_table <- result$updated_table

# # Diagnostics (list)
  # result$diagnostics$summary # needing manual review just means have never been checked


# Write the updated table including diagnostic columns on the right
write.csv(updated_table, "./annotated-new_output_comparisson/Table1_matched_strict_2025-12-10_reviewing.cs
```

```r
knitr::knit_exit()
```